```
myValue = 4 + 38;
```

Beginning on the right side, the Arduino will add 4 and 38 and assign the sum to the variable `myValue`, which now will equal the numerical value of 42. Likewise, since variables act just like the numbers that they store, they can be included in the equation as well.

```
myValue = 255;
newValue = myValue - 128;
```

After assigning the value 255 to `myValue`, the next line subtracts 128 from `myValue` and assigns the result to `newValue` that now contains the number 127. We can even do this using the same variable as in the following example:

```
myValue = 255;
myValue = myValue - 1;
```

Beginning with 255, 1 is subtracted from `myValue` and this new result, 254, is reassigned to the variable `myValue`. If we performed this second line 255 times in a row, we would **decrement**, or lower the value of `myValue` until it reached 0.

The values that we are performing these operations on are called **operands**. The data type of the operands determines the data type used by the operators with the largest type being used if they are of different types. Consider the following code fragment, which should be somewhat familiar from high school geometry:

```
float pi = 3.14159;
int diameter = 5;
float C;
C = pi * diameter;
```

In this case, the operator used to determine `C` ignores the fact that the variable `diameter` is an integer data type and, in the last line, we multiply the float variable `pi` (3.14159) to the integer variable `diameter` (5) and we end up with the floating-point value of 15.7079505920 assigned to the variable `C`. This shows that when one of the operands is of the float data type, floating point math is used for the entire calculation and, as a result, suffers from the usual drawbacks of large memory size and slow operation.

In this most recent example, the variable `C` was of the float data type and as a result could store the fractional value. If, however, both of the operands are of the same data type, the operation will be performed using that type regardless of the actual outcome, as follows, for example:

```
int myValue;
myValue = 9 / 5;
```

This statement divides 9 by 5; however, since `myValue` is an integer type variable, the resulting value assigns the lobbed off result of 1 to `myValue` rather than the actual value of 1.8 because integers can only contain whole numbers. Notice that this isn't rounding up or down but rather disregarding any fractional numbers after the decimal point.

Be careful about accidently dividing a number by zero, as in `x = 9 / 0;`. Rather than giving you a compiler error for something you should not be able to do, the Arduino compiler will instead spit out some rather odd and unexpected numbers.